



```
In [ ]: from google.colab import drive
drive.mount('/content/drive')

import os
os.chdir('/content/drive/MyDrive')
```

Mounted at /content/drive

```
In [ ]: import pandas as pd
df = pd.read_csv('zomato.csv', encoding='latin1')
df.head()
```

Out[]:

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Locality_Ve
0	6317637	Le Petit Souffle	162	Makati City	Third Floor, Century City Mall, Kalayaan Avenue...	Century City Mall, Poblacion, Makati City	Century City Mall, Poblacion, Makati City
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, 2277 Chino Roces Avenue, Legaspi...	Little Tokyo, Legaspi Village, Makati City	Little Tokyo, Legaspi Village, Makati City
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, 1 Garden Way, Ortigas, Mandal...	Edsa Shangri-La, Ortigas, Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City
3	6318506	Ooma	162	Mandaluyong City	Third Floor, Mega Fashion Hall, SM Megamall, O...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City
4	6314302	Sambo Kojin	162	Mandaluyong City	Third Floor, Mega Atrium, SM Megamall, Ortigas...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City

5 rows × 21 columns

```
In [ ]: country_df = pd.read_excel("Country-Code.xlsx")
country_df.head()
```

```
Out[ ]:
```

	Country Code	Country
0	1	India
1	14	Australia
2	30	Brazil
3	37	Canada
4	94	Indonesia

```
In [ ]: df = df.merge(
    country_df,
    on="Country Code",
    how="left"
)
```

```
In [ ]: pd.set_option('display.max_columns', None)
df.head()
```

Out[]:

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Locality Verbose
0	6317637	Le Petit Souffle	162	Makati City	Third Floor, Century City Mall, Kalayaan Avenue...	Century City Mall, Poblacion, Makati City	Century City Mall, Poblacion, Makati City
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, 2277 Chino Roces Avenue, Legaspi...	Little Tokyo, Legaspi Village, Makati City	Little Tokyo, Legaspi Village, Makati City
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, 1 Garden Way, Ortigas, Mandal...	Edsa Shangri-La, Ortigas, Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City
3	6318506	Ooma	162	Mandaluyong City	Third Floor, Mega Fashion Hall, SM Megamall, O...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City
4	6314302	Sambo Kojin	162	Mandaluyong City	Third Floor, Mega Atrium, SM Megamall, Ortigas...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City

In []: `df.columns`

Out[]: Index(['Restaurant ID', 'Restaurant Name', 'Country Code', 'City', 'Address', 'Locality', 'Locality Verbose', 'Longitude', 'Latitude', 'Cuisines', 'Average Cost for two', 'Currency', 'Has Table booking', 'Has Online delivery', 'Is delivering now', 'Switch to order menu', 'Price range', 'Aggregate rating', 'Rating color', 'Rating text', 'Votes', 'Country'], dtype='object')

In []: `df[['Switch to order menu', 'Rating text']].value_counts()`

Out[]: **count**

Switch to order menu		Rating text	
	No	Average	3737
		Not rated	2148
		Good	2100
		Very Good	1079
		Excellent	301
		Poor	186

dtype: int64

```
In [ ]: df[['Is delivering now']].value_counts()
```

Out[]: **count**

Is delivering now		
No		9517
Yes		34

dtype: int64

```
In [ ]: df['Has Online delivery'].value_counts()
```

Out[]: **count**

Has Online delivery		
No		7100
Yes		2451

dtype: int64

```
In [ ]: df['Has Table booking'].value_counts()
```

Out[]: **count**

Has Table booking	
No	8393
Yes	1158

dtype: int64

```
In [ ]: df['Rating text'].value_counts()
```

Out[]: **count**

Rating text	
Average	3737
Not rated	2148
Good	2100
Very Good	1079
Excellent	301
Poor	186

dtype: int64

```
In [ ]: df['Currency'].value_counts()
```

Out[]:

	count
Currency	
Indian Rupees(Rs.)	8652
Dollar(\$)	482
Pounds(£)	80
Brazilian Real(R\$)	60
Rand(R)	60
Emirati Diram(AED)	60
NewZealand(\$)	40
Turkish Lira(TL)	34
Botswana Pula(P)	22
Indonesian Rupiah>IDR)	21
Qatari Rial(QR)	20
Sri Lankan Rupee(LKR)	20

dtype: int64

In []:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9551 entries, 0 to 9550
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Restaurant ID                        9551 non-null   int64
1   Restaurant Name                      9551 non-null   object
2   Country Code                        9551 non-null   int64
3   City                                9551 non-null   object
4   Address                             9551 non-null   object
5   Locality                           9551 non-null   object
6   Locality Verbose                    9551 non-null   object
7   Longitude                           9551 non-null   float64
8   Latitude                            9551 non-null   float64
9   Cuisines                            9542 non-null   object
10  Average Cost for two                 9551 non-null   int64
11  Currency                            9551 non-null   object
12  Has Table booking                   9551 non-null   object
13  Has Online delivery                 9551 non-null   object
14  Is delivering now                   9551 non-null   object
15  Switch to order menu                 9551 non-null   object
16  Price range                         9551 non-null   int64
17  Aggregate rating                    9551 non-null   float64
18  Rating color                        9551 non-null   object
19  Rating text                         9551 non-null   object
20  Votes                              9551 non-null   int64
21  Country                             9551 non-null   object
dtypes: float64(3), int64(5), object(14)
memory usage: 1.6+ MB
```

```
In [ ]: df.drop(columns=['Switch to order menu','Rating color'],inplace=True)
```

```
In [ ]: df.duplicated().sum()
```

```
Out[ ]: np.int64(0)
```

```
In [ ]: df.isna().sum().sum()
# Cuisines column has 9 missing values
```

```
Out[ ]: np.int64(9)
```

```
In [ ]: # Admiring How can we fill the missing values
df[df['Cuisines'].isna()]['City'].value_counts()
```

Out[]: **count**

City	
Albany	3
Gainesville	1
Macon	1
Miller	1
Orlando	1
Rest of Hawaii	1
Savannah	1

dtype: int64

```
In [ ]: df[(df['City'] == 'Rest of Hawaii')]['Cuisines'].value_counts()
```

Out[]: **count**

Cuisines	
Hawaiian, Seafood, Steak	3
Seafood	2
American, Hawaiian	1
American, Pizza, Bar Food	1
Japanese, Seafood, Sushi	1
Asian	1
Asian, Breakfast, Hawaiian	1
Burger	1
Breakfast, Burger	1
Hawaiian, Seafood	1
Japanese	1
American, Asian, Burger	1
American	1
Breakfast	1
Asian, European, Seafood	1
American, Breakfast, Hawaiian	1

dtype: int64


```
In [ ]: df[(df['City'] == 'Gainesville')]['Cuisines'].value_counts()
```

```
Out[ ]:
```

	count
Cuisines	
BBQ	2
American, Southern	2
American, Sandwich, Seafood	1
American, Burger, Cajun	1
BBQ, Burger, Seafood	1
Chinese, Japanese, Thai	1
Bar Food, Seafood, Vegetarian	1
Seafood	1
BBQ, Sandwich	1
Sandwich, Seafood, Cajun	1
Thai	1
American, European, Sandwich	1
Southern	1
German	1
German, Ice Cream	1
Burger, German, Sandwich	1
Italian, Pizza, Sandwich	1

dtype: int64

```
In [ ]: df[(df['City'] == 'Albany')]['Cuisines'].value_counts()
```

Out[]: **count**

Cuisines	
Japanese, Steak, Sushi	2
BBQ, Burger, Seafood	1
Mexican	1
American, BBQ	1
Fast Food	1
Asian, Chinese, Vegetarian	1
Pizza, Bar Food, Sandwich	1
Coffee and Tea, Sandwich	1
Steak, Tapas, Bar Food	1
Chinese, Seafood, Vegetarian	1
American, Burger, Sandwich	1
Chinese	1
American, Steak	1
Seafood, Tapas, Bar Food	1
Italian, Pizza	1
American, Breakfast, Diner	1

dtype: int64

```
In [ ]: # As the Cuisines corresponding to their City is not a good option to replacin  
# So, replacing the values with 'Fast Food' for Cuisine column
```

```
In [ ]: df['Cuisines'].value_counts()
```

Out[]:

	count
Cuisines	
North Indian	936
North Indian, Chinese	511
Chinese	354
Fast Food	354
North Indian, Mughlai	334
...	...
World Cuisine, Patisserie, Cafe	1
Burger, Izgara	1
Desserts, B́rek	1
Restaurant Cafe, Turkish, Desserts	1
Restaurant Cafe, Desserts	1

1825 rows × 1 columns

dtype: int64

```
In [ ]: df['Cuisines'] = df['Cuisines'].fillna('Fast Food')  
# Missing Values DONE!
```

```
In [ ]: df.sample(5)
```

Out[]:

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Locality Verbose
7714	18416856	Captain Chang	1	New Delhi	Vasant Kunj, New Delhi	Vasant Kunj	Vasant Kunj, New Delhi
2606	18354658	Punjabi Chaap Corner	1	New Delhi	Shop 6, GF, Plot 2, NRI Colony, Alaknanda, New...	Alaknanda	Alaknanda, New Delhi
3631	18421462	Prem Ji Delhi Wale	1	New Delhi	6/174, Near Ramleela Ground, Geeta Colony, New...	Geeta Colony	Geeta Colony, New Delhi
2598	7639	Madhuvan Chinese Fast Food	1	New Delhi	Alaknanda Shopping Complex, Near Gangotri Comp...	Alaknanda	Alaknanda, New Delhi
2026	306976	KB's Kulfi & Icecream	1	Gurgaon	1, Opposite Allahabad Bank, Sector 7 Chowk, Se...	Sector 7	Sector 7, Gurgaon

Analysing Address like columns

```
In [ ]: df.iloc[0]['Address']
```

```
Out[ ]: 'Third Floor, Century City Mall, Kalayaan Avenue, Poblacion, Makati City'
```

```
In [ ]: df.iloc[0]['Locality Verbose']
```

```
Out[ ]: 'Century City Mall, Poblacion, Makati City, Makati City'
```

```
In [ ]: df.iloc[0]['Locality']
```

```
Out[ ]: 'Century City Mall, Poblacion, Makati City'
```

```
In [ ]: df.iloc[100]['Address']
```

```
Out[ ]: '1604 N Slappey Blvd, Albany, GA 31701'
```

```
In [ ]: df.iloc[100]['Locality Verbose']
```

```
Out[ ]: 'Albany, Albany'
```

```
In [ ]: df.iloc[100]['Locality']
```

```
Out[ ]: 'Albany'
```

```
In [ ]: df.iloc[100]['Country']
```

```
Out[ ]: 'United States'
```

```
In [ ]: df.iloc[55]['Address']
```

```
Out[ ]: 'Avenida Ataulfo de Paiva, 1022, Lojas A e B, Leblon, Rio de Janeiro'
```

```
In [ ]: df['Address'].nunique()
```

```
Out[ ]: 8918
```

```
In [ ]: df['Locality'].nunique()
```

```
Out[ ]: 1208
```

```
In [ ]: df['Locality Verbose'].nunique()
```

```
Out[ ]: 1265
```

```
In [ ]: df['Country'].nunique()
```

```
Out[ ]: 15
```

```
In [ ]: df.drop(columns=['Address', 'Locality Verbose'], inplace=True)  
df.head()
```

Out[]:

	Restaurant ID	Restaurant Name	Country Code	City	Locality	Longitude	Latitude
0	6317637	Le Petit Souffle	162	Makati City	Century City Mall, Poblacion, Makati City	121.027535	14.56
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, Legaspi Village, Makati City	121.014101	14.55
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City	121.056831	14.56
3	6318506	Ooma	162	Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City	121.056475	14.56
4	6314302	Sambo Kojin	162	Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City	121.057508	14.56

In []: df.sample(5)

Out[]:

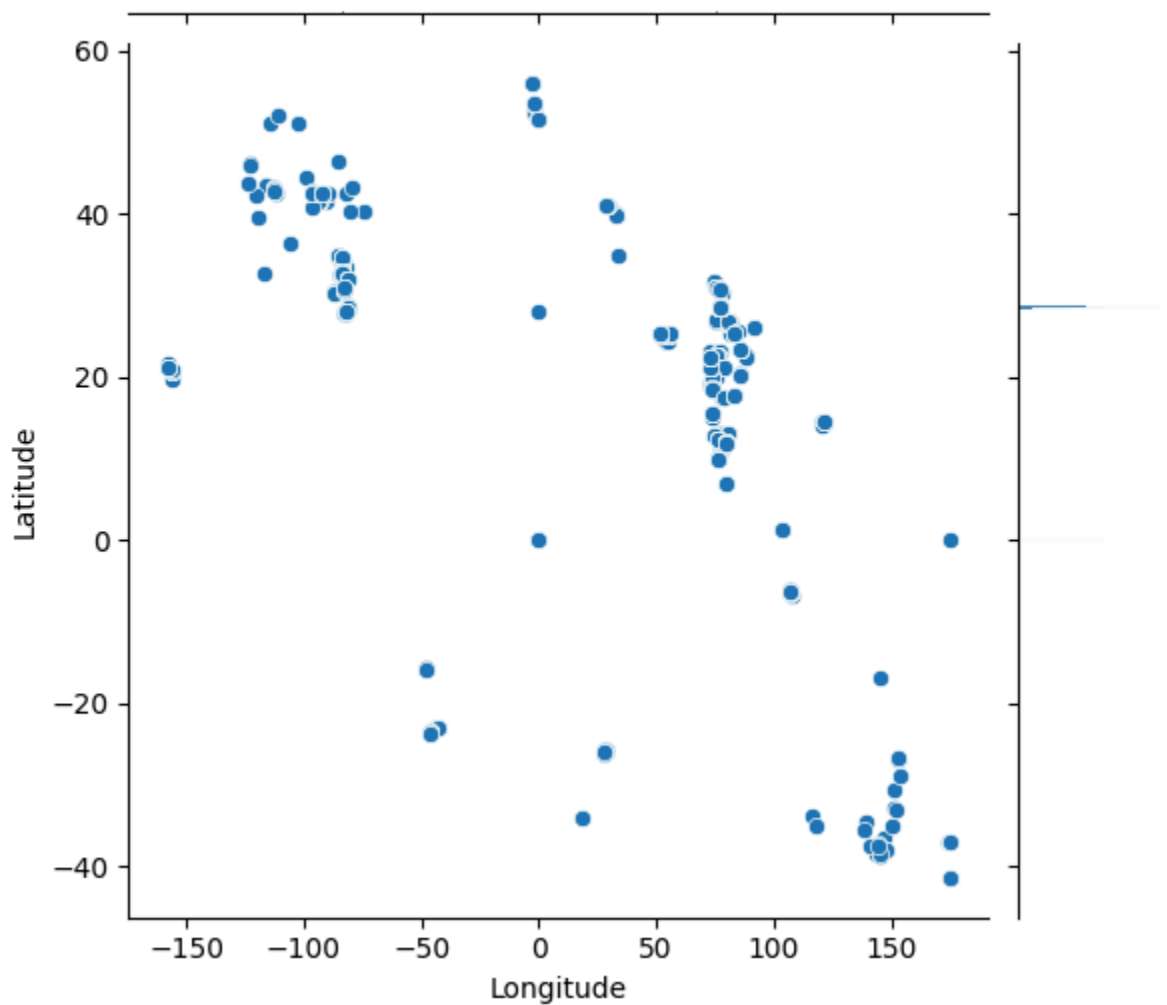
	Restaurant ID	Restaurant Name	Country Code	City	Locality	Longitude	Latitude
1477	307304	The Cinnamon Kitchen	1	Gurgaon	Golf Course Road	77.097410	28.4370
9327	7100072	Fisherman's Plate	148	Wellington City	Te Aro	174.775667	-41.2890
5724	304753	Baketown By Gagan Bakers	1	New Delhi	Nangloi	77.054728	28.6823
5747	304782	Ramesh Caterers & Dhaba	1	New Delhi	Nangloi	77.072183	28.6536
4812	18485936	Chez Jerome - Q Cafe	1	New Delhi	Lado Sarai	0.000000	0.0000

EDA

```
In [ ]: import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [ ]: sns.jointplot(x = 'Longitude', y = 'Latitude', data=df)
```

```
Out[ ]: <seaborn.axisgrid.JointGrid at 0x7aa23919c200>
```



```
In [ ]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9551 entries, 0 to 9550
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Restaurant ID                        9551 non-null   int64
1   Restaurant Name                      9551 non-null   object
2   Country Code                        9551 non-null   int64
3   City                                9551 non-null   object
4   Locality                            9551 non-null   object
5   Longitude                           9551 non-null   float64
6   Latitude                            9551 non-null   float64
7   Cuisines                            9551 non-null   object
8   Average Cost for two                9551 non-null   int64
9   Currency                            9551 non-null   object
10  Has Table booking                   9551 non-null   object
11  Has Online delivery                 9551 non-null   object
12  Is delivering now                   9551 non-null   object
13  Price range                         9551 non-null   int64
14  Aggregate rating                    9551 non-null   float64
15  Rating text                         9551 non-null   object
16  Votes                              9551 non-null   int64
17  Country                             9551 non-null   object
dtypes: float64(3), int64(5), object(10)
memory usage: 1.3+ MB

```

```
In [ ]: df.select_dtypes(include=['int64','int32','float64']).corr()
```

```
Out[ ]:
```

	Restaurant ID	Country Code	Longitude	Latitude	Average Cost for two	Price range	Aggregate rating	Votes
Restaurant ID	1.000000	0.148471	-0.226081	-0.052081	-0.001693	-0.134540	-0.326212	-0.147023
Country Code	0.148471	1.000000	-0.698299	0.019792	0.043225	0.243327	0.282189	0.154530
Longitude	-0.226081	-0.698299	1.000000	0.043207	0.045891	-0.078939	-0.116818	-0.085101
Latitude	-0.052081	0.019792	0.043207	1.000000	-0.111088	-0.166688	0.000516	-0.022962
Average Cost for two	-0.001693	0.043225	0.045891	-0.111088	1.000000	0.075083	0.051792	0.067783
Price range	-0.134540	0.243327	-0.078939	-0.166688	0.075083	1.000000	0.437944	0.309444
Aggregate rating	-0.326212	0.282189	-0.116818	0.000516	0.051792	0.437944	1.000000	0.908800
Votes	-0.147023	0.154530	-0.085101	-0.022962	0.067783	0.309444	0.908800	1.000000

```
In [ ]: # To see the correlation matrix without sorting:
# df.corr(numeric_only=True)
```



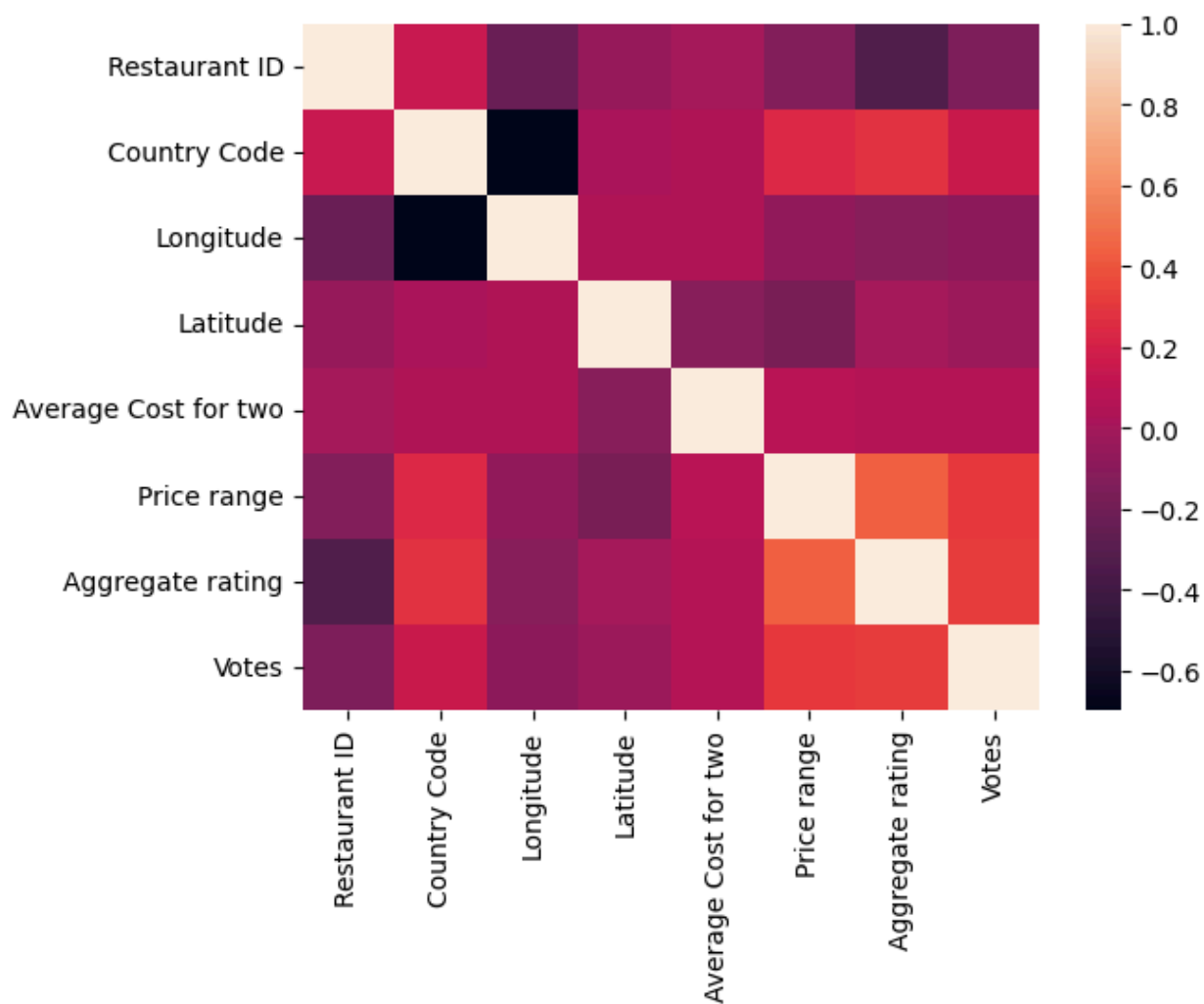
```
# Or, to sort the correlation matrix by a specific column, for example, 'Aggregate rating'
df.corr(numeric_only=True).sort_values(by='Aggregate rating', ascending=False)
```

Out[]:

	Restaurant ID	Country Code	Longitude	Latitude	Average Cost for two	Price range	Aggregate rating
Aggregate rating	-0.326212	0.282189	-0.116818	0.000516	0.051792	0.437944	1.000000
Price range	-0.134540	0.243327	-0.078939	-0.166688	0.075083	1.000000	0.437944
Votes	-0.147023	0.154530	-0.085101	-0.022962	0.067783	0.309444	0.437944
Country Code	0.148471	1.000000	-0.698299	0.019792	0.043225	0.243327	0.437944
Average Cost for two	-0.001693	0.043225	0.045891	-0.111088	1.000000	0.075083	0.437944
Latitude	-0.052081	0.019792	0.043207	1.000000	-0.111088	-0.166688	0.437944
Longitude	-0.226081	-0.698299	1.000000	0.043207	0.045891	-0.078939	0.437944
Restaurant ID	1.000000	0.148471	-0.226081	-0.052081	-0.001693	-0.134540	0.437944

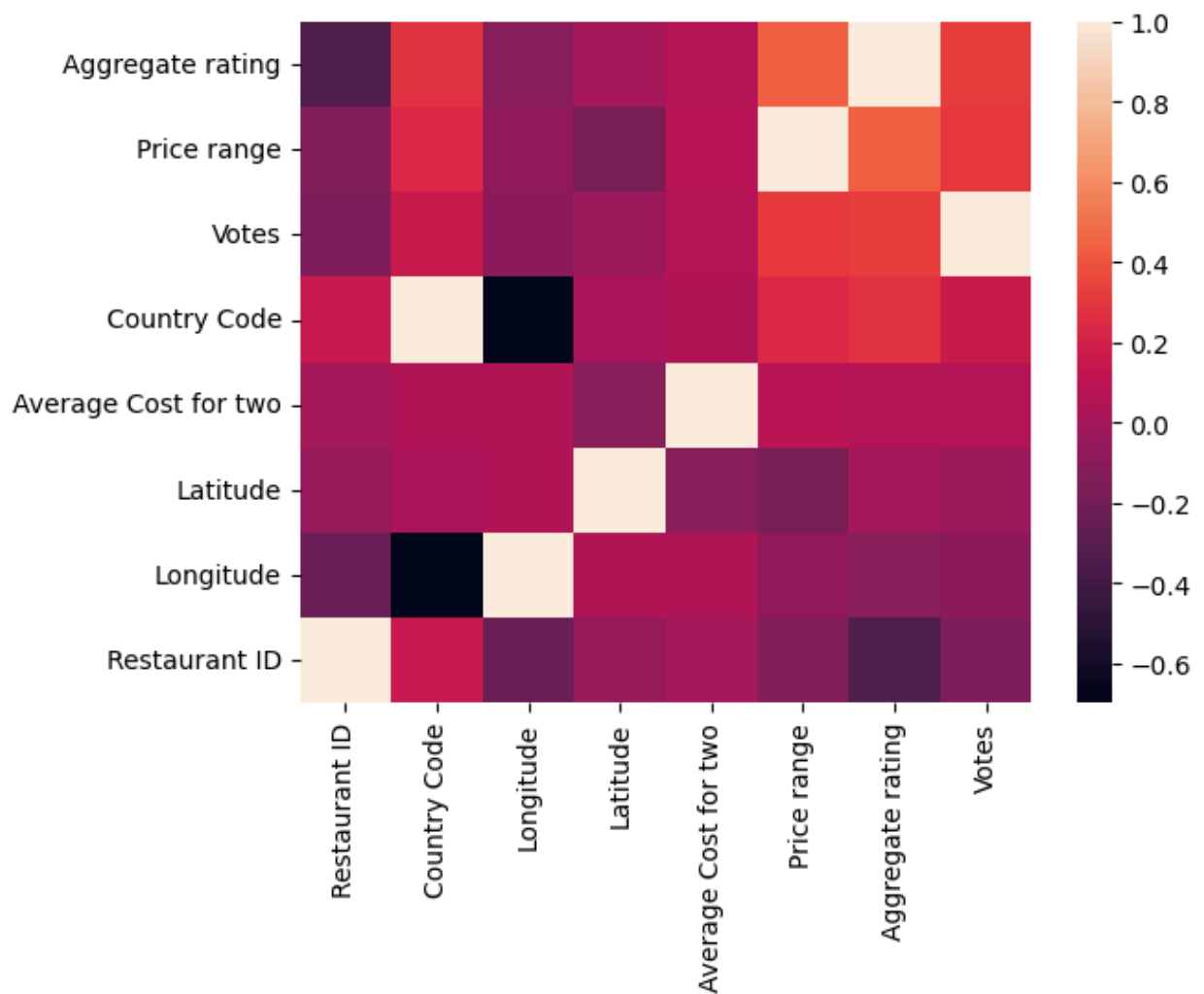
```
In [ ]: sns.heatmap(df.select_dtypes(include=['int64','int32','float64']).corr())
```

Out[]: <Axes: >



```
In [ ]: sns.heatmap(df.corr(numeric_only=True).sort_values(by='Aggregate rating', asce
```

```
Out[ ]: <Axes: >
```



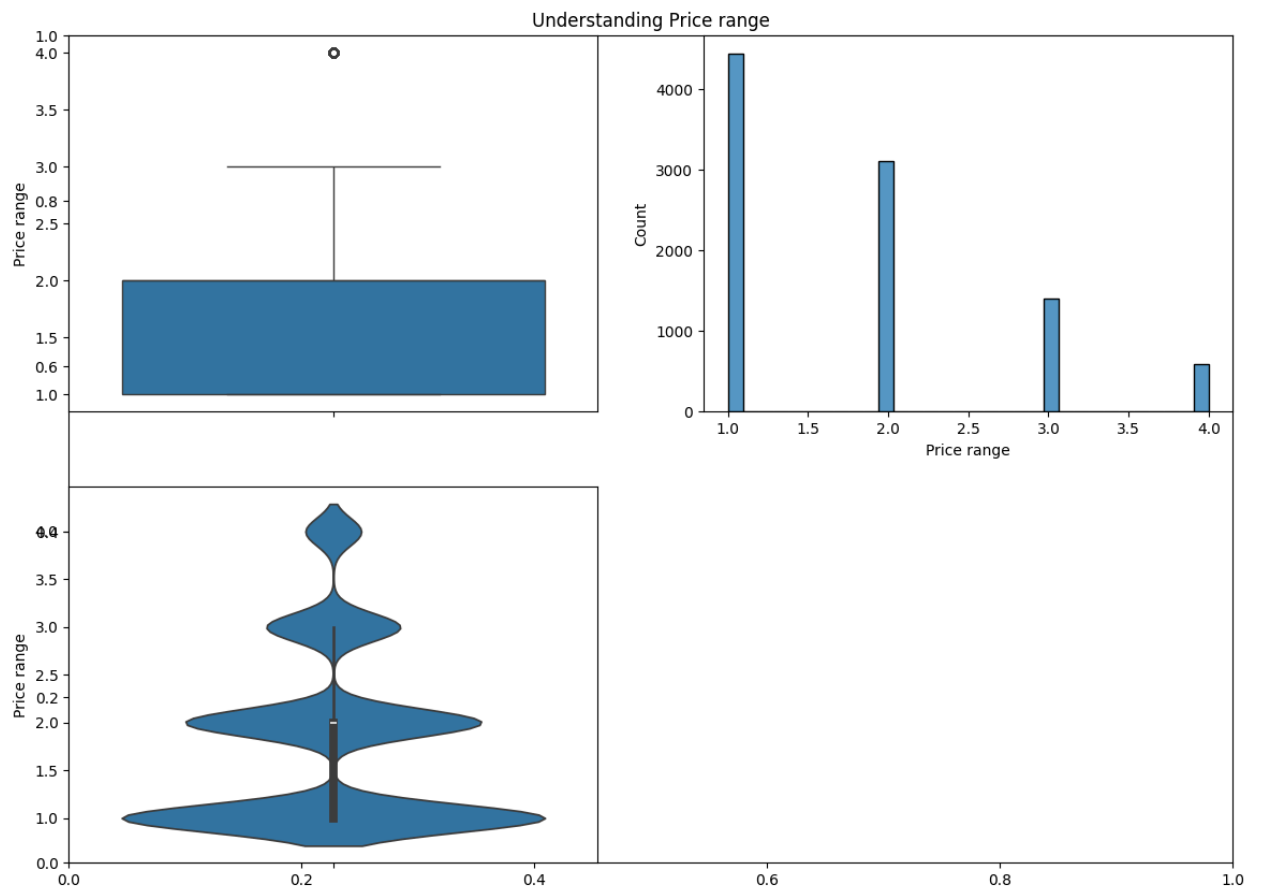
Univariate Analysis

```
In [ ]: plt.figure(figsize=(14,10))
plt.title("Understanding Price range")
plt.subplot(2,2,1)
sns.boxplot(df['Price range'])

plt.subplot(2,2,2)
sns.histplot(df['Price range'])

plt.subplot(2,2,3)
sns.violinplot(df['Price range'])
```

```
Out[ ]: <Axes: ylabel='Price range'>
```



```
In [ ]: df[df['Price range'] > 3].count()
```

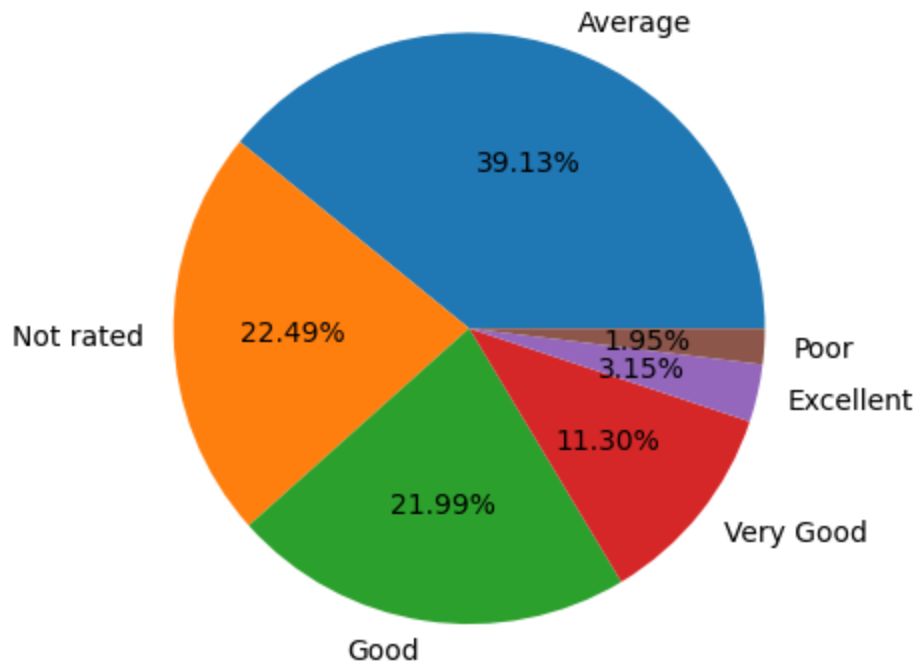
Out[]: 0

Restaurant ID	586
Restaurant Name	586
Country Code	586
City	586
Locality	586
Longitude	586
Latitude	586
Cuisines	586
Average Cost for two	586
Currency	586
Has Table booking	586
Has Online delivery	586
Is delivering now	586
Price range	586
Aggregate rating	586
Rating text	586
Votes	586
Country	586

dtype: int64

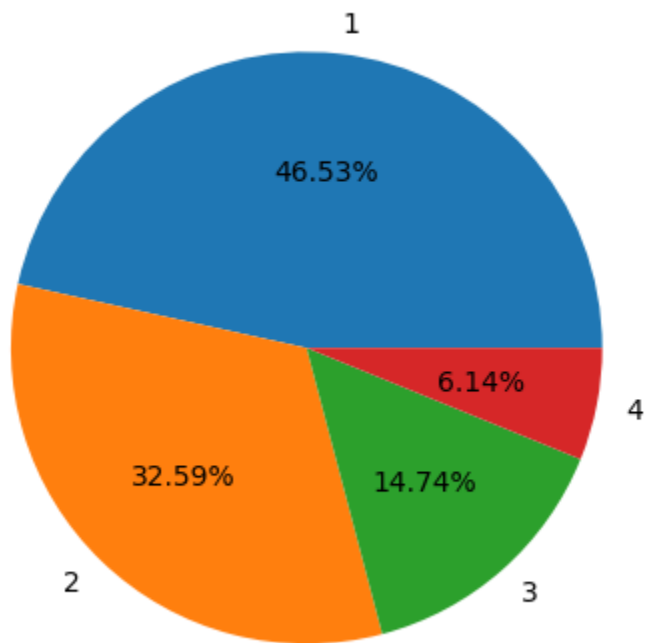
```
In [ ]: plt.title("Rating Text Distribution")
plt.pie(df['Rating text'].value_counts(), labels=df['Rating text'].value_count
plt.show()
```

Rating Text Distribution



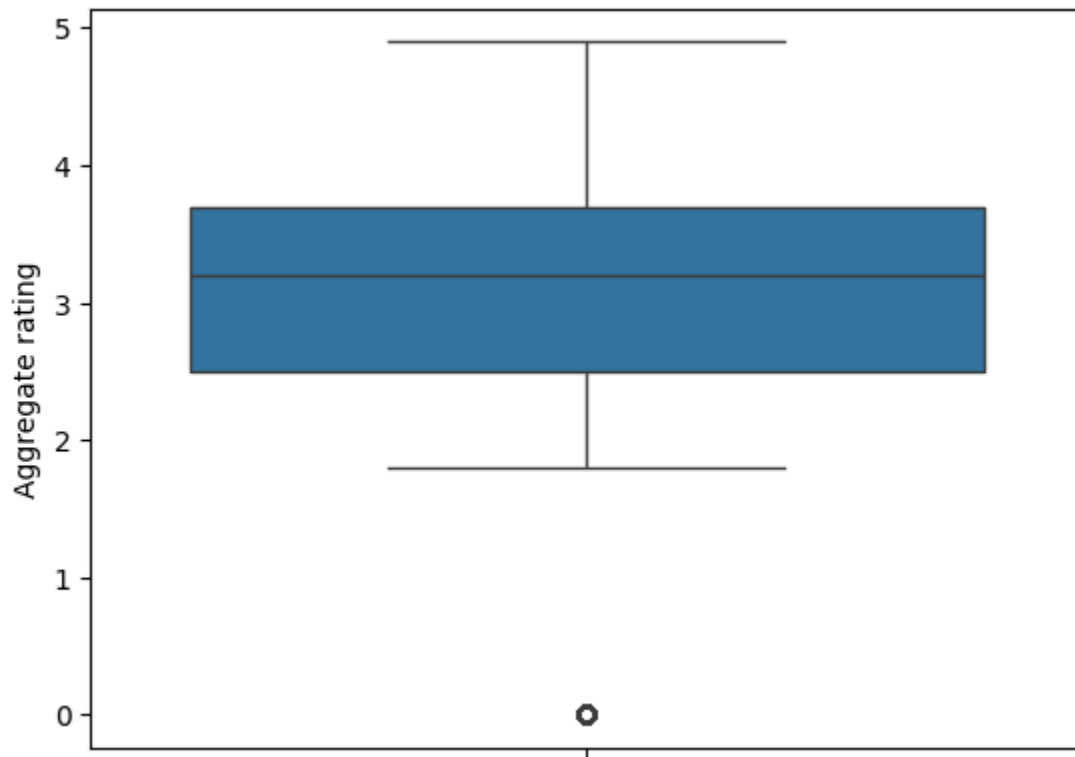
```
In [ ]: plt.title("Price range Distribution")
plt.pie(df['Price range'].value_counts(), labels=df['Price range'].value_count
plt.show()
```

Price range Distribution



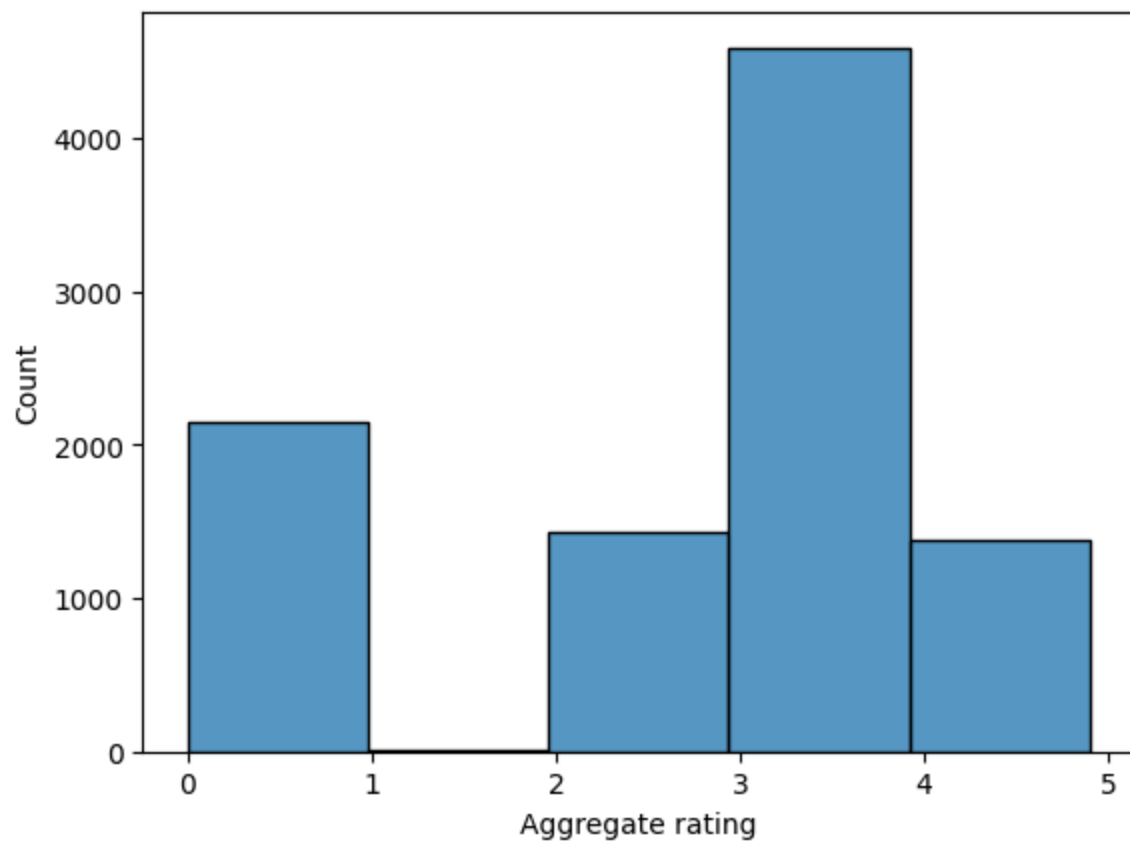
```
In [ ]: sns.boxplot(df['Aggregate rating'])
```

```
Out[ ]: <Axes: ylabel='Aggregate rating'>
```



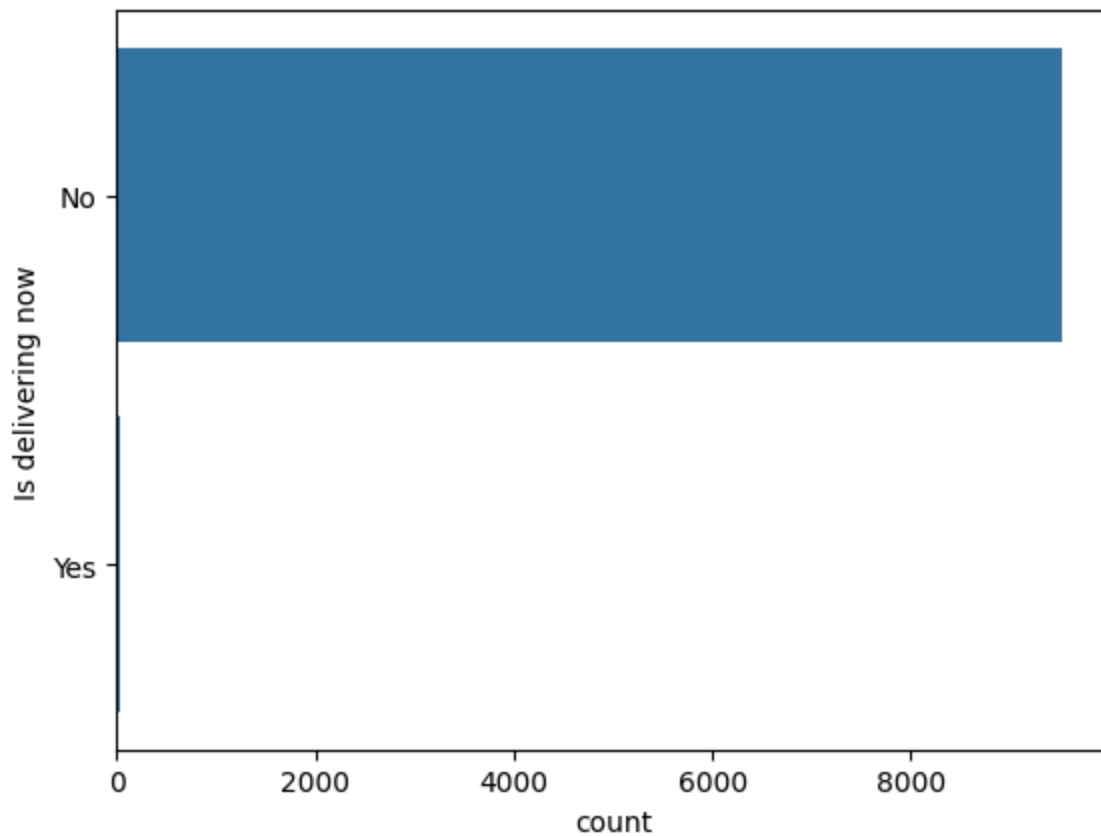
```
In [ ]: sns.histplot(df['Aggregate rating'], bins=5)
```

```
Out[ ]: <Axes: xlabel='Aggregate rating', ylabel='Count'>
```



```
In [ ]: sns.countplot(df['Is delivering now'])
```

```
Out[ ]: <Axes: xlabel='count', ylabel='Is delivering now'>
```

```
In [ ]: country_df.sample(5)
```

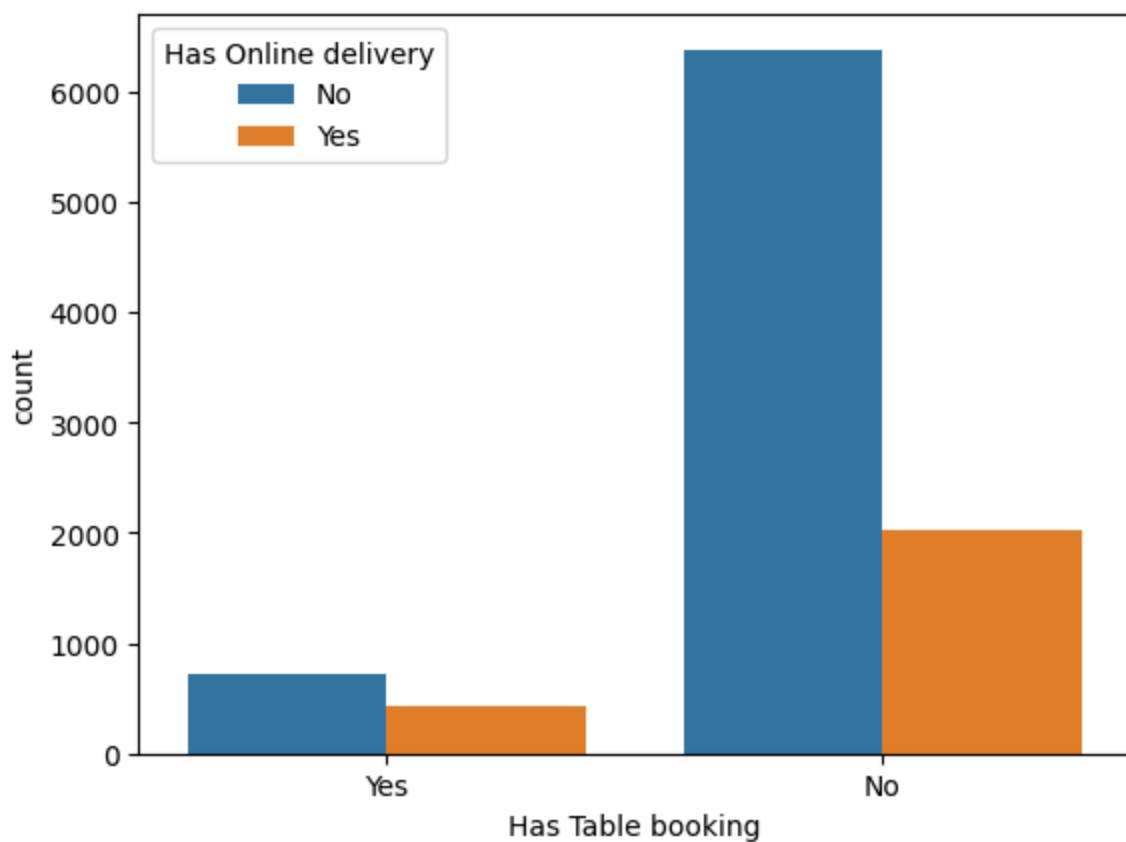
```
Out[ ]:
```

	Country Code	Country
3	37	Canada
4	94	Indonesia
2	30	Brazil
10	191	Sri Lanka
11	208	Turkey

Multivariate Analysis

```
In [ ]: sns.countplot(data=df, x = 'Has Table booking', hue='Has Online delivery')
```

```
Out[ ]: <Axes: xlabel='Has Table booking', ylabel='count'>
```



```
In [ ]: df.columns
```

```
Out[ ]: Index(['Restaurant ID', 'Restaurant Name', 'Country Code', 'City', 'Localit
y',
              'Longitude', 'Latitude', 'Cuisines', 'Average Cost for two', 'Currenc
y',
              'Has Table booking', 'Has Online delivery', 'Is delivering now',
              'Price range', 'Aggregate rating', 'Rating text', 'Votes', 'Country'],
              dtype='object')
```

```
In [ ]: ratings = df.groupby(['Aggregate rating', 'Rating text']).size().reset_index().
```

```
In [ ]: ratings.head()
```

```
Out[ ]:
```

	Aggregate rating	Rating text	Rating count
0	0.0	Not rated	2148
1	1.8	Poor	1
2	1.9	Poor	2
3	2.0	Poor	7
4	2.1	Poor	15

Rating count vs Aggregate rating

- Most of the people Not rated the Order
- Most of the people who Not rated the Order are from India (It can be also because Most of the users are from India)

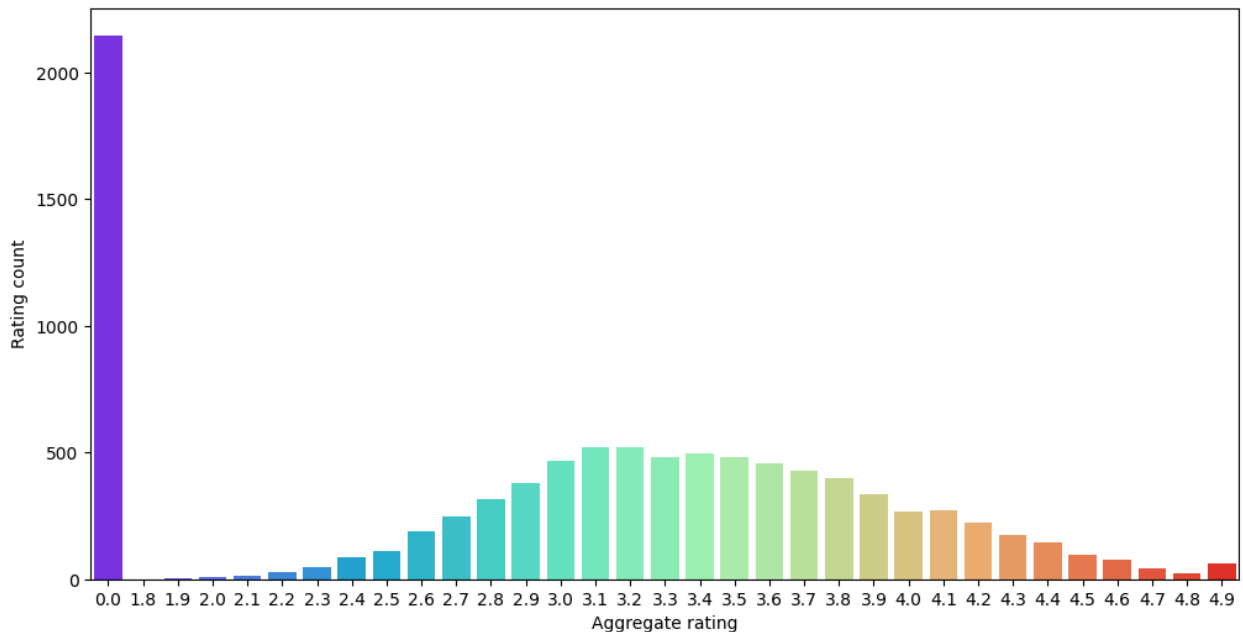
```
In [ ]: plt.rcParams['figure.figsize'] = (12,6)
sns.barplot(x = 'Aggregate rating', y = 'Rating count', data=ratings, palette=
```

/tmp/ipython-input-2027444791.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

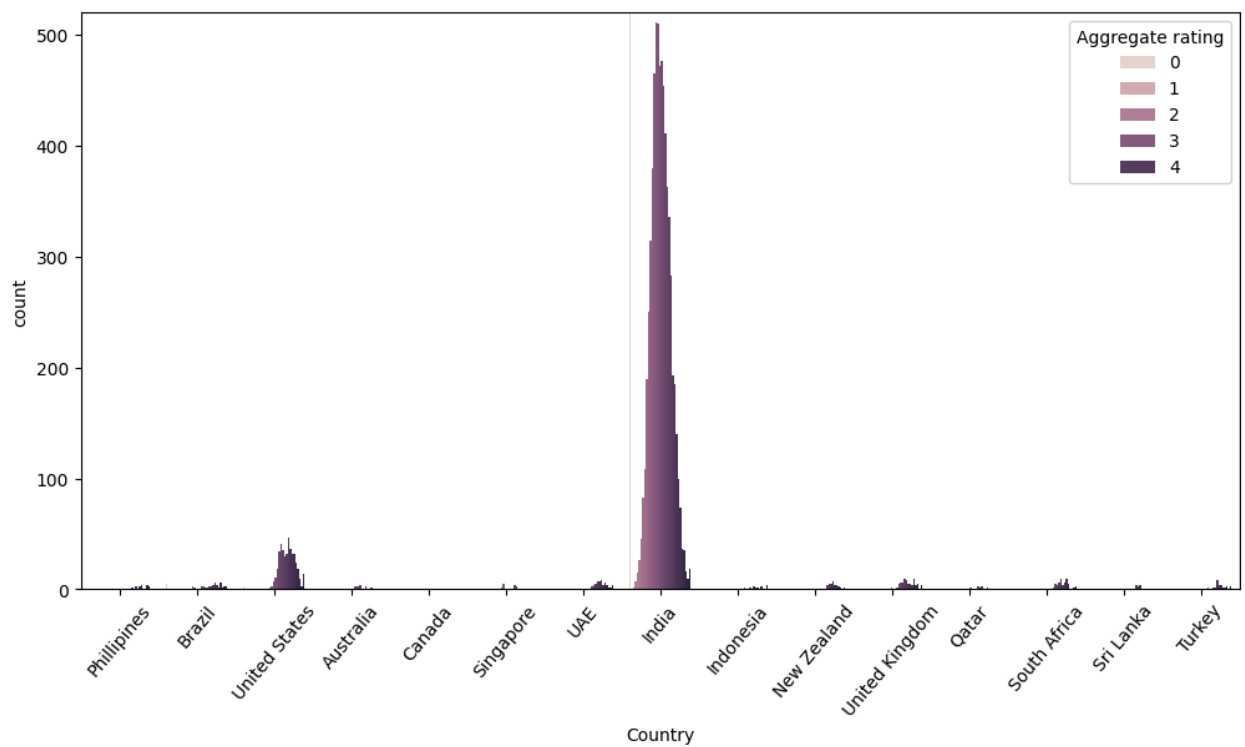
```
sns.barplot(x = 'Aggregate rating', y = 'Rating count', data=ratings, palette='rainbow')
```

```
Out[ ]: <Axes: xlabel='Aggregate rating', ylabel='Rating count'>
```



```
In [ ]: sns.countplot(x = 'Country', hue = 'Aggregate rating', data=df)
plt.xticks(rotation=50)
plt.ylim(0, 520)
```

```
Out[ ]: (0.0, 520.0)
```



```
In [ ]: ratings['Aggregate rating'].skew()
```

```
Out[ ]: np.float64(-0.6625533094099731)
```

```
In [ ]: df.groupby(['Aggregate rating', 'Country']).size().sort_index().reset_index().h
```

```
Out[ ]:
```

	Aggregate rating	Country	0
0	0.0	Brazil	5
1	0.0	India	2139
2	0.0	United Kingdom	1
3	0.0	United States	3
4	1.8	India	1

Model Building

```
In [ ]: df.head(3)
```

	Restaurant ID	Restaurant Name	Country Code	City	Locality	Longitude	Latitude
0	6317637	Le Petit Souffle	162	Makati City	Century City Mall, Poblacion, Makati City	121.027535	14.56
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, Legaspi Village, Makati City	121.014101	14.55
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City	121.056831	14.56

```
In [ ]: df[['City', 'Locality', 'Longitude', 'Latitude', 'Cuisines', 'Currency', 'Country']]
```

```
Out[ ]:
0
City      141
Locality  1208
Longitude  8120
Latitude  8677
Cuisines  1825
Currency   12
Country    15
```

dtype: int64

```
In [ ]: # Percentage of Unique cities in the whole DF
df['City'].nunique() / len(df) * 100
```

```
Out[ ]: 1.476285205737619
```

```
In [ ]: df.groupby(['Country', 'Currency']).size()

# Both are same so removing one(Currency)
```

Out[]:

0

Country	Currency	
Australia	Dollar(\$)	24
Brazil	Brazilian Real(R\$)	60
Canada	Dollar(\$)	4
India	Indian Rupees(Rs.)	8652
Indonesia	Indonesian Rupiah(IDR)	21
New Zealand	NewZealand(\$)	40
Phillipines	Botswana Pula(P)	22
Qatar	Qatari Rial(QR)	20
Singapore	Dollar(\$)	20
South Africa	Rand(R)	60
Sri Lanka	Sri Lankan Rupee(LKR)	20
Turkey	Turkish Lira(TL)	34
UAE	Emirati Diram(AED)	60
United Kingdom	Pounds(£)	80
United States	Dollar(\$)	434

dtype: int64

```
In [ ]: # Dropping high cardinality columns
df.drop(columns=['Locality', 'Cuisines', 'Restaurant Name', 'Country Code', 'Curre
```

```
In [ ]: df['Average Individual Cost'] = df['Average Cost for two'] / 2
```

```
In [ ]: df.head()
```

Out[]:

	Restaurant ID	City	Longitude	Latitude	Average Cost for two	Has Table booking	Has Online delivery
0	6317637	Makati City	121.027535	14.565443	1100	Yes	No
1	6304287	Makati City	121.014101	14.553708	1200	Yes	No
2	6300002	Mandaluyong City	121.056831	14.581404	4000	Yes	No
3	6318506	Mandaluyong City	121.056475	14.585318	1500	No	No
4	6314302	Mandaluyong City	121.057508	14.584450	1500	Yes	No

In []: `df.drop('Average Cost for two',axis=1, inplace=True)`

In []: `df.head()`

Out[]:

	Restaurant ID	City	Longitude	Latitude	Has Table booking	Has Online delivery	Is delivering now
0	6317637	Makati City	121.027535	14.565443	Yes	No	No
1	6304287	Makati City	121.014101	14.553708	Yes	No	No
2	6300002	Mandaluyong City	121.056831	14.581404	Yes	No	No
3	6318506	Mandaluyong City	121.056475	14.585318	No	No	No
4	6314302	Mandaluyong City	121.057508	14.584450	Yes	No	No

In []:

```
plt.figure(figsize=(12,6))

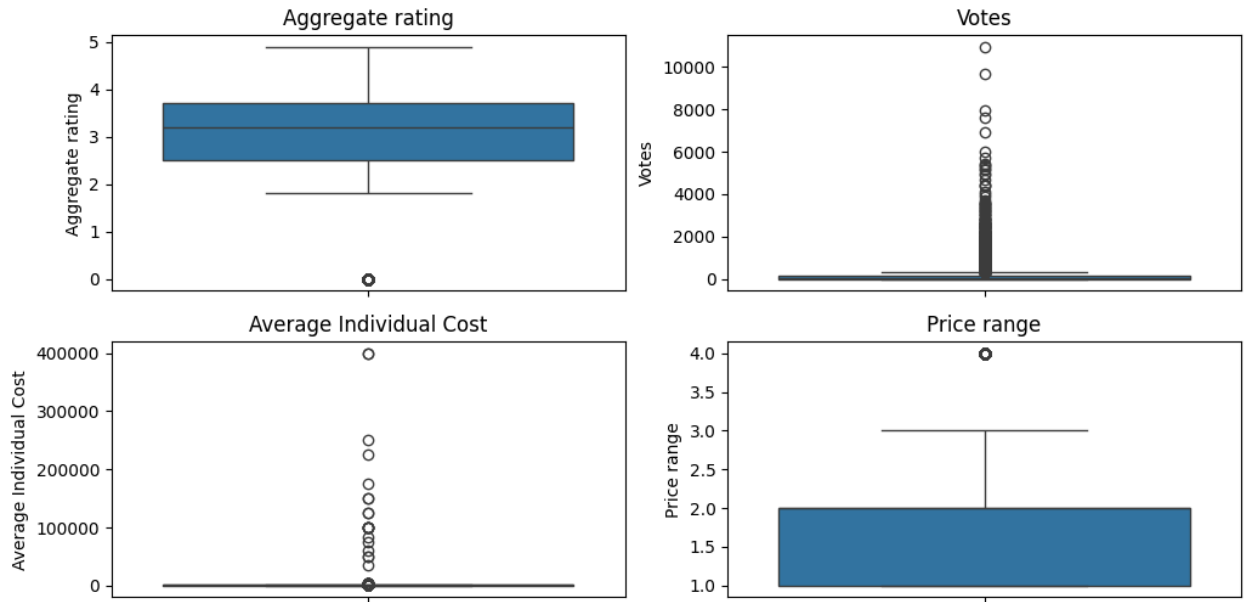
plt.subplot(2,2,1)
plt.title('Aggregate rating')
sns.boxplot(df['Aggregate rating'])

plt.subplot(2,2,2)
plt.title('Votes')
sns.boxplot(df['Votes'])

plt.subplot(2,2,3)
plt.title('Average Individual Cost')
sns.boxplot(df['Average Individual Cost'])

plt.subplot(2,2,4)
plt.title('Price range')
sns.boxplot(df['Price range'])
```

Out[]: <Axes: title={'center': 'Price range'}, ylabel='Price range'>



```
In [ ]: from sklearn.preprocessing import PowerTransformer, OneHotEncoder, OrdinalEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from sklearn.ensemble import RandomForestRegressor
```

```
In [ ]: X = df.drop(columns=['Aggregate rating'])
y = df['Aggregate rating']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
In [ ]: X_train.head()
```

Out[]:

	Restaurant ID	City	Longitude	Latitude	Has Table booking	Has Online delivery	Is delivering now
1115	8417	Faridabad	77.306073	28.472912	Yes	No	No
7633	301912	New Delhi	77.039089	28.621372	No	No	No
8267	18424175	Noida	77.387116	28.533194	No	No	No
7209	18418252	New Delhi	77.282282	28.632808	No	No	No
6242	2958	New Delhi	77.135169	28.688001	No	No	No


```
In [ ]: y_train.head()
```

```
Out[ ]: Aggregate rating
```

1115	3.3
7633	3.0
8267	0.0
7209	0.0
6242	2.5

dtype: float64

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9551 entries, 0 to 9550
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Restaurant ID                        9551 non-null  int64
1   City                                9551 non-null  object
2   Longitude                           9551 non-null  float64
3   Latitude                           9551 non-null  float64
4   Has Table booking                   9551 non-null  object
5   Has Online delivery                 9551 non-null  object
6   Is delivering now                   9551 non-null  object
7   Price range                         9551 non-null  int64
8   Aggregate rating                    9551 non-null  float64
9   Rating text                         9551 non-null  object
10  Votes                              9551 non-null  int64
11  Country                            9551 non-null  object
12  Average Individual Cost              9551 non-null  float64
dtypes: float64(4), int64(3), object(6)
memory usage: 970.2+ KB
```

```
In [ ]: df.head()
```

Out[]:

	Restaurant ID	City	Longitude	Latitude	Has Table booking	Has Online delivery	Is delivering now
0	6317637	Makati City	121.027535	14.565443	Yes	No	No
1	6304287	Makati City	121.014101	14.553708	Yes	No	No
2	6300002	Mandaluyong City	121.056831	14.581404	Yes	No	No
3	6318506	Mandaluyong City	121.056475	14.585318	No	No	No
4	6314302	Mandaluyong City	121.057508	14.584450	Yes	No	No

In []:

Starts

Learned the Pipeline concept with ColumnTransformer

In []: `df.head()`

Out[]:

	Restaurant ID	City	Longitude	Latitude	Has Table booking	Has Online delivery	Is delivering now
0	6317637	Makati City	121.027535	14.565443	Yes	No	No
1	6304287	Makati City	121.014101	14.553708	Yes	No	No
2	6300002	Mandaluyong City	121.056831	14.581404	Yes	No	No
3	6318506	Mandaluyong City	121.056475	14.585318	No	No	No
4	6314302	Mandaluyong City	121.057508	14.584450	Yes	No	No

In []: `num_power_transformer_cols = ['Votes', 'Average Individual Cost', 'Longitude', 'Latitude']`
`num_scale_cols = ['Price range', 'Votes', 'Average Individual Cost', 'Longitude', 'Latitude']`
`cat_oh_cols = ['Has Table booking', 'Has Online delivery', 'Is delivering now']`
`cat_ordinal_cols = ['Rating text']`

In []: `ordinal_mapping = ['Not rated', 'Poor', 'Average', 'Good', 'Very Good', 'Excellent']`

In []: `power_transform_pipeline = Pipeline([`
 `('power_transform', PowerTransformer()),`
 `('scaler', StandardScaler())`
`])`

```

])

scale_pipeline = Pipeline([
    ('scaler', StandardScaler())
])

ohe_pipeline = Pipeline([
    ('one_hot_encoder', OneHotEncoder(handle_unknown='ignore', drop='first'))
])

ordinal_pipeline = Pipeline([
    ('ordinal_encoder', OrdinalEncoder(categories=[ordinal_mapping]))
])

```

```

In [ ]: preprocessor = ColumnTransformer(transformers=[
    ('num_power_scale', power_transform_pipeline, num_power_transformer_cols),
    ('num_scale', scale_pipeline, num_scale_cols),
    ('cat_ohe', ohe_pipeline, cat_ohe_cols),
    ('ordinal', ordinal_pipeline, cat_ordinal_cols)
])

```

```

In [ ]: final_pipeline = Pipeline(steps=[('preprocessor', preprocessor)])

```

```

In [ ]: X_train_transformed = final_pipeline.fit_transform(X_train)
        X_test_transformed = final_pipeline.transform(X_test)

```

/usr/local/lib/python3.12/dist-packages/sklearn/preprocessing/_encoders.py:246:
UserWarning: Found unknown categories in columns [4] during transform. These un
known categories will be encoded as all zeros
warnings.warn(

```

In [ ]: lr = LinearRegression()
        lr.fit(X_train_transformed, y_train)

```

```

Out[ ]: ▼ LinearRegression ⓘ ?
        LinearRegression()

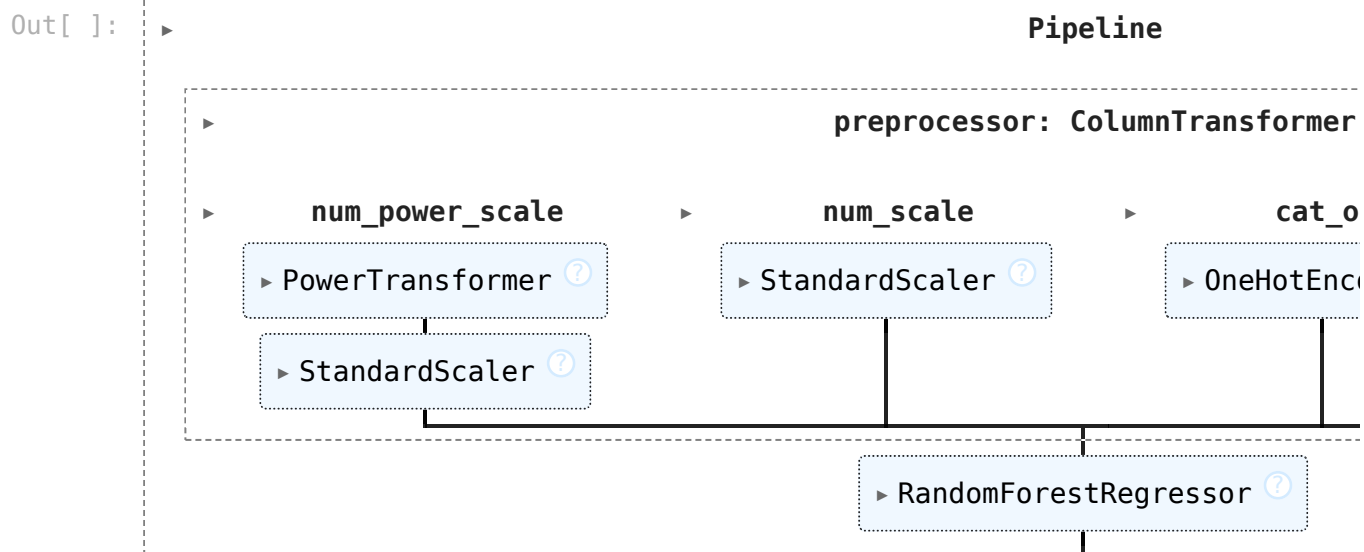
```

```

In [ ]: rf_regressor = RandomForestRegressor(random_state=42)

# Create a pipeline for Random Forest Regressor using the same preprocessor
rf_pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('regressor', rf
rf_pipeline.fit(X_train, y_train)

```



```

In [ ]: y_pred = lr.predict(X_test_transformed)
        r2 = r2_score(y_test, y_pred) * 100
        print(f"R2 score for Linear Regression: {r2:.2f}%")

        y_pred_rf = rf_pipeline.predict(X_test)
        r2_rf = r2_score(y_test, y_pred_rf) * 100
        print(f"R2 score for Random Forest Regressor: {r2_rf:.2f}%")

```

R2 score for Linear Regression: 92.35%

R2 score for Random Forest Regressor: 98.75%

/usr/local/lib/python3.12/dist-packages/sklearn/preprocessing/_encoders.py:246:
UserWarning: Found unknown categories in columns [4] during transform. These unknown categories will be encoded as all zeros
warnings.warn(

```

In [ ]: # Without Ordinal Mapping: 79.72
        # After Ordinal Mapping: 91.87
        # After Adding 'Longitude, Latitude': 92.19
        # After Adding 'Restaurant ID' : 92.28
        # After Adding 'City' : 92.34

        # Random Forest model : 98.75 (RandomForest takes more time to train as compared to Linear Regression)

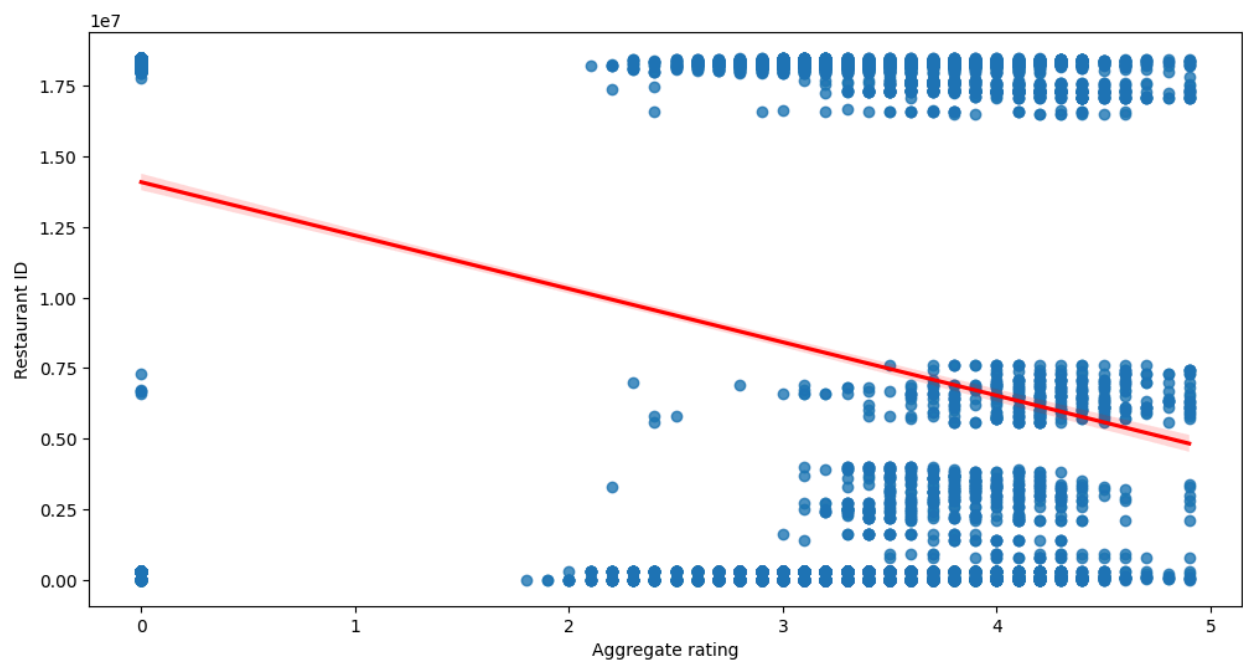
```

```

In [ ]: sns.regplot(x = 'Aggregate rating', y='Restaurant ID', data=df, line_kws={'color': 'red'})

```

Out []: <Axes: xlabel='Aggregate rating', ylabel='Restaurant ID'>



Manual testing

```
In [ ]: df.sample(1)
```

```
Out[ ]:
```

	Has Table booking	Has Online delivery	Is delivering now	Price range	Aggregate rating	Rating text	Votes	Country
2317	Yes	No	No	3	4.1	Very Good	571	India

```
In [ ]: sample = pd.DataFrame(X_train_transformed)
```

```
In [ ]: sample.head()
```

```
Out[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10
0	0.348060	1.523105	1.315988	-0.232512	0.047166	1.0	0.0	0.0	0.0	0.0	1.0
1	-0.844028	-1.240740	-0.892987	-0.354481	-0.071644	0.0	0.0	0.0	0.0	0.0	1.0
2	-1.622012	-1.240740	-0.892987	-0.363863	-0.071644	0.0	0.0	0.0	0.0	0.0	1.0
3	-1.287959	-0.431972	-0.892987	-0.361518	-0.061743	0.0	0.0	0.0	0.0	0.0	1.0
4	0.490241	0.384539	0.211501	-0.187946	-0.038641	0.0	0.0	0.0	0.0	0.0	1.0

```
In [ ]: X_train_transformed
```

```
Out[ ]: array([[ 0.34806005,  1.52310466,  1.3159881 , ...,  0.          ,
                 0.          ,  2.          ],
                [-0.84402789, -1.24074001, -0.89298676, ...,  0.          ,
                 0.          ,  2.          ],
                [-1.62201194, -1.24074001, -0.89298676, ...,  0.          ,
                 0.          ,  0.          ],
                ...,
                [-1.62201194, -0.4319717 , -0.89298676, ...,  0.          ,
                 0.          ,  0.          ],
                [ 0.42791474,  1.34874439,  2.42047553, ...,  0.          ,
                 0.          ,  4.          ],
                [ 0.96036398,  0.38453906,  0.21150067, ...,  0.          ,
                 0.          ,  3.          ]])
```

```
In [ ]: import numpy as np
        lr.predict(np.array([-0.844028, -1.240740, -0.892987, -0.354481, -0.071644, 0.0, 0.0]))
```

```
Out[ ]: array([2.38857159])
```

```
In [ ]: df.head(2)
```

```
Out[ ]:
```

	Has Table booking	Has Online delivery	Is delivering now	Price range	Aggregate rating	Rating text	Votes	Country	I
0	Yes	No	No	3	4.8	Excellent	314	Phillipines	
1	Yes	No	No	3	4.5	Excellent	591	Phillipines	

Data-Driven Insights

- A huge chunk of restaurants were never rated by customers, especially in India.
- Only a few restaurants allow table booking, usually the fancier ones.
- At this time, the Online delivery was not so common as today on Zomato
- The majority fall in the lower price ranges (1 or 2 out of 4).
- Only 9 rows had no cuisine listed. Since no clear pattern existed, "Fast Food" was used as a safe default.